



1D Finite Difference

Python Tutorial for CFD

1 1D Linear Convection

In this tutorial, we will explore the finite difference method for solving the 1D Linear Convection equation:

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0$$

The convection equation models the propagation of a wave with speed c , and its solution represents how the initial wave $u(x, 0) = u_0(x)$ moves over time. The exact solution is given by:

$$u(x, t) = u_0(x - ct)$$

We will discretize this equation using finite differences and solve it numerically in Python.

1.1 Discretization

The time derivative $\frac{\partial u}{\partial t}$ will be approximated using a Forward Difference scheme, and the spatial derivative $\frac{\partial u}{\partial x}$ will be approximated using a Backward Difference scheme.

$$\frac{\partial u}{\partial x} \approx \frac{u(x_i) - u(x_{i-1})}{\Delta x}$$

Substituting these into the convection equation, we get:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + c \frac{u_i^n - u_{i-1}^n}{\Delta x} = 0$$

Rearranging the terms, we can solve for u_i^{n+1} :

$$u_i^{n+1} = u_i^n - c \frac{\Delta t}{\Delta x} (u_i^n - u_{i-1}^n)$$

1.2 Python Implementation

Now, we will implement this discretized equation in Python. First, we need to import necessary libraries.

1.2.1 Step 1: Importing Libraries

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3 import time, sys
4 %matplotlib inline
```

1.2.2 Step 2: Defining the Grid

Next, we define the spatial domain and the time steps. Let $nx = 41$ be the number of grid points, and $dx = \frac{2}{nx-1}$ is the distance between adjacent grid points.

```
1 nx = 41 # number of spatial points
2 dx = 2 / (nx - 1) # spacing between points
3 nt = 25 # number of time steps
4 dt = 0.025 # time step size
5 c = 1 # wave speed
```

1.2.3 Step 3: Initial Condition

We now set the initial condition where $u = 2$ between $0.5 \leq x \leq 1$ and $u = 1$ elsewhere.

```
1 u = np.ones(nx) # initialize u with all ones
2 u[int(0.5 / dx):int(1 / dx + 1)] = 2 # set u = 2 between 0.5 and 1
3 print(u)
```

1.2.4 Step 4: Visualizing Initial Condition

We can visualize the initial condition using a simple plot.

```
1 plt.plot(np.linspace(0, 2, nx), u)
2 plt.show()
```

1.2.5 Step 5: Finite Difference Loop

We now implement the finite difference scheme to advance the solution in time. We'll use a nested loop, where the outer loop iterates over time and the inner loop updates the values of u at each spatial point using the discretized equation.

```

1 un = np.ones(nx) # initialize a temporary array
2
3 for n in range(nt): # loop over time steps
4     un = u.copy() # copy the old u values into un
5     for i in range(1, nx):
6         u[i] = un[i] - c * dt / dx * (un[i] - un[i-1])
7
8 plt.plot(np.linspace(0, 2, nx), u)
9 plt.show()

```

Conclusion: In this tutorial, we have solved the 1D linear convection equation using the finite difference method. The numerical method discretized the equation using Forward Difference in time and Backward Difference in space. The Python implementation shows how the wave moves to the right as time progresses, but note that the solution is not perfect due to numerical dispersion.

To improve the accuracy of the solution, we will explore more advanced techniques in future tutorials.

2 Step 2: Nonlinear Convection

We now turn to the 1D nonlinear convection equation:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0$$

Unlike the linear convection equation, where c was constant, here the convection speed depends on u itself, which makes the problem nonlinear.

2.1 Discretization

We will apply the same discretization method as in Step 1: forward difference for the time derivative and backward difference for the spatial derivative. The discretized equation is:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + u_i^n \frac{u_i^n - u_{i-1}^n}{\Delta x} = 0$$

Solving for u_i^{n+1} , we get:

$$u_i^{n+1} = u_i^n - u_i^n \frac{\Delta t}{\Delta x} (u_i^n - u_{i-1}^n)$$

Now, we will implement this discretization in Python.

2.2 Python Implementation

We begin by importing the necessary libraries and defining our grid, as we did in Step 1.

```

1 import numpy as np
2 from matplotlib import pyplot as plt
3 %matplotlib inline
4
5 nx = 41 # number of grid points
6 dx = 2 / (nx - 1) # spatial step size
7 nt = 20 # number of time steps
8 dt = 0.025 # time step size
9
10 u = np.ones(nx) # initialize u with ones
11 u[int(0.5 / dx) : int(1 / dx + 1)] = 2 # set the initial condition
12 print(u)

```

2.3 Time-Stepping the Nonlinear Convection Equation

We will now modify the code from Step 1 to implement the nonlinear convection equation. The main difference is that the convection term is now multiplied by u_i^n , making the update equation nonlinear.

```

1 un = np.ones(nx) # initialize a temporary array
2
3 for n in range(nt): # loop over time
4     un = u.copy() # copy the values of u into un
5     for i in range(1, nx): # loop over space
6         u[i] = un[i] - un[i] * dt / dx * (un[i] - un[i-1])
7
8 plt.plot(np.linspace(0, 2, nx), u)
9 plt.show()

```

2.4 Results and Observations

You should observe that the initial hat function $u(x, 0)$ changes over time, and its evolution under the nonlinear convection equation is quite different from the linear case. Try changing the number of grid points nx , time steps nt , and time step size dt to see how they affect the solution.

What happens to the shape of the hat function under nonlinear convection? What do you observe when the hat steepens into a shock? Nonlinear terms introduce phenomena such as steepening of gradients, which are not seen in linear convection.

Conclusion In this step, we implemented the 1D nonlinear convection equation using the finite difference method. The nonlinearity arises because the wave speed depends on the solution u , which leads to different behavior than the linear case. Nonlinear convection can result in sharp gradients or shocks, which can be challenging to resolve numerically.

Experiment with different numerical parameters and observe the changes in the wave's behavior. In the next steps, we will explore methods to improve stability and accuracy.

3 Diffusion Equation in 1D

The one-dimensional diffusion equation is given by:

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2}$$

This equation models how a quantity u diffuses over time. Unlike the previous equations, this one contains a second-order spatial derivative. We'll discretize this equation using finite differences and solve it numerically.

3.1 Discretizing the Second Derivative

We use a Central Difference scheme to discretize the second derivative. By taking the Taylor expansions of u_{i+1} and u_{i-1} around u_i , we can derive the following discretization for the second derivative:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}$$

Substituting this into the diffusion equation gives:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

Solving for u_i^{n+1} :

$$u_i^{n+1} = u_i^n + \nu \frac{\Delta t}{\Delta x^2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

3.2 Python Implementation

We will now implement this equation in Python using the same structure as the previous steps.

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3 %matplotlib inline
4
5 # Set up the grid and parameters
6 nx = 41 # number of grid points
7 dx = 2 / (nx - 1) # spatial step size
8 nt = 20 # number of time steps
9 nu = 0.3 # viscosity
10 sigma = 0.2 # stability parameter
11 dt = sigma * dx**2 / nu # time step size
12
13 # Initial condition: hat function
14 u = np.ones(nx)
```

```

15 u[int(0.5 / dx) : int(1 / dx + 1)] = 2
16
17 # Initialize a temporary array for time-stepping
18 un = np.ones(nx)
19
20 # Time-stepping loop
21 for n in range(nt): # iterate through time
22     un = u.copy() # copy the old values of u into un
23     for i in range(1, nx - 1): # update all interior points
24         u[i] = un[i] + nu * dt / dx**2 * (un[i+1] - 2 * un[i] + un[i-1])
25
26 # Plot the result
27 plt.plot(np.linspace(0, 2, nx), u)
28 plt.show()

```

3.3 Results and Observations

The initial condition for this problem is a "hat function", just as in the previous steps. You should observe that, over time, the sharp edges of the hat smooth out, which is characteristic of diffusion processes.

Try experimenting with different values of nt , dx , nu , and $sigma$ to see how they affect the diffusion rate and the stability of the solution. A smaller time step (determined by dt) can help ensure stability, but may require more iterations to see the effect of diffusion.

Conclusion: In this step, we implemented the 1D diffusion equation using the finite difference method. The diffusion equation smooths out the initial condition over time, which is expected from a diffusive process. The discretization of the second derivative required us to use a Central Difference scheme, which we implemented in the code.

In the next steps, we will look into more complex systems that combine both convection and diffusion, as well as methods for improving numerical stability and accuracy.

4 Step 4: Burgers' Equation

Burgers' equation is a fundamental partial differential equation combining both nonlinear convection and diffusion. The 1D Burgers' equation is written as:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

This equation can teach us a lot about shock formation and wave propagation. We can discretize it using the methods we've already explored for convection and diffusion.

4.1 Discretization

We apply forward difference in time, backward difference in space, and central difference for the second derivative. The discretized form of Burgers' equation becomes:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + u_i^n \frac{u_i^n - u_{i-1}^n}{\Delta x} = \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

Rearranging, we can solve for u_i^{n+1} :

$$u_i^{n+1} = u_i^n - u_i^n \frac{\Delta t}{\Delta x} (u_i^n - u_{i-1}^n) + \nu \frac{\Delta t}{\Delta x^2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

5 Initial and Boundary Conditions

For this problem, we will use a periodic boundary condition $u(0) = u(2\pi)$, and the initial condition is derived from an analytical solution to Burgers' equation:

$$u(x, 0) = -2\nu \frac{\partial \phi}{\partial x} + 4$$

where:

$$\phi = e^{-\frac{(x-4t)^2}{4\nu(t+1)}} + e^{-\frac{(x-4t-2\pi)^2}{4\nu(t+1)}}$$

5.1 Using SymPy for Initial Conditions

To simplify the symbolic math, we will use SymPy to compute the derivative of ϕ and set up the initial condition for our simulation.

```

1 import numpy as np
2 import sympy as sp
3 from sympy import init_printing
4 init_printing(use_latex=True)
5
6 # Define symbolic variables
7 x, nu, t = sp.symbols('x nu t')
8 phi = (sp.exp(-(x - 4 * t)**2 / (4 * nu * (t + 1))) +
9         sp.exp(-(x - 4 * t - 2 * sp.pi)**2 / (4 * nu * (t + 1))))
10 phi
```

$$e^{-\frac{(x-4t)^2}{4\nu(t+1)}} + e^{-\frac{(x-4t-2\pi)^2}{4\nu(t+1)}}$$

Now we compute the derivative of ϕ with respect to x :

```

1 # Compute the derivative of phi
2 phiprime = phi.diff(x)
3 phiprime
```

The result is:

$$-\frac{(-8t+2x)e^{-\frac{(x-4t)^2}{4\nu(t+1)}}}{4\nu(t+1)} - \frac{(-8t+2x-4\pi)e^{-\frac{(x-4t-2\pi)^2}{4\nu(t+1)}}}{4\nu(t+1)}$$

We then use `lambdify` to create a Python function from this symbolic expression.

```
1 from sympy.utilities.lambdify import lambdify
2
3 # Create a Python function for u
4 u = -2 * nu * (phiprime / phi) + 4
5 ufunc = lambdify((t, x, nu), u)
6
7 # Example: Evaluate u at t=1, x=4, nu=3
8 print(ufunc(1, 4, 3))
```

6 Setting up the Burgers' Equation Solver

We will now implement the solver for Burgers' equation using the discretized form derived earlier. The initial condition is generated using the function created above.

```
1 from matplotlib import pyplot as plt
2 %matplotlib inline
3
4 # Set up grid and parameters
5 nx = 101
6 nt = 100
7 dx = 2 * np.pi / (nx - 1)
8 nu = 0.07
9 dt = dx * nu
10
11 x = np.linspace(0, 2 * np.pi, nx)
12 un = np.empty(nx)
13 t = 0
14
15 # Generate the initial condition
16 u = np.asarray([ufunc(t, x0, nu) for x0 in x])
17
18 # Plot the initial condition
19 plt.figure(figsize=(11, 7), dpi=100)
20 plt.plot(x, u, marker='o', lw=2)
21 plt.xlim([0, 2 * np.pi])
22 plt.ylim([0, 10])
23 plt.title('Initial Condition for Burgers\' Equation')
24 plt.show()
```

7 Time-stepping with Periodic Boundary Conditions

Now we will solve Burgers' equation using the time-stepping method, implementing the periodic boundary conditions carefully.

```
1 # Time-stepping loop
2 for n in range(nt):
```

```

3     un = u.copy()
4     for i in range(1, nx-1):
5         u[i] = un[i] - un[i] * dt / dx * (un[i] - un[i-1]) + nu * dt / dx**2 * \
6             (un[i+1] - 2 * un[i] + un[i-1])
7     u[0] = un[0] - un[0] * dt / dx * (un[0] - un[-2]) + nu * dt / dx**2 * \
8         (un[1] - 2 * un[0] + un[-2])
9     u[-1] = u[0] # Enforce periodic boundary condition
10
11 # Compute the analytical solution for comparison
12 u_analytical = np.asarray([ufunc(nt * dt, xi, nu) for xi in x])
13
14 # Plot the results
15 plt.figure(figsize=(11, 7), dpi=100)
16 plt.plot(x, u, marker='o', lw=2, label='Computational')
17 plt.plot(x, u_analytical, label='Analytical')
18 plt.xlim([0, 2 * np.pi])
19 plt.ylim([0, 10])
20 plt.legend()
21 plt.title('Burgers\' Equation: Computational vs. Analytical Solution')
22 plt.show()

```

Conclusion In this step, we solved Burgers' equation, combining both nonlinear convection and diffusion. We used periodic boundary conditions to simulate wave propagation around a circular domain. We also compared the computational solution with the analytical solution for validation.

Experiment with different parameters such as the number of time steps nt , the grid resolution nx , and the viscosity ν to observe different behaviors of the solution.